SkipQ: A Real-Time Geofenced Micro-Ordering and Queue Mitigation System for Campus Canteens

Satyam Patel
Dept. of Computer Engineering
Silver Oak University, India
satyam.patel@silveroakuni.ac.in

Harsh V. Trivedi
Dept. of Computer Engineering
Silver Oak University, India
harsh.trivedi@silveroakuni.ac.in

Priya R. Shah
Dept. of Information Technology
Silver Oak University, India
priya.shah@silveroakuni.ac.in

Abstract* - *Traditional university dining facilities suffer from acute counter congestion during synchronized lecture breaks, creating extensive queue wait times, lost instructional hours, and operational overhead for cafeteria operators. Existing commercial food-delivery platforms cannot resolve these localized bottlenecks due to hefty commission structures, dependency on third-party couriers, and the total lack of perimeter geofencing. This paper presents SkipQ, an Expo-based, real-time micro-ordering and queue-mitigation application for higher education campuses. The current implementation uses client-side Haversine distance checks with a 500-meter ordering perimeter and a bounded GPS-accuracy tolerance; it connects buyer ordering with a seller Point-of-Sale (POS) workflow through Cloud Firestore when configured. The application also includes a Dynamic Active Booking Pill, a four-digit pickup-PIN handover flow, wallet and cancellation handling, and configurable anti-ghosting controls. This paper documents the implemented architecture and defines a reproducible evaluation plan; it does not claim unmeasured deployment outcomes.

Index Terms* - *Campus dining, Geofencing, Mobile ordering, Queue mitigation, Real-time state synchronization.

I. INTRODUCTION

University campus environments exhibit extreme, synchronized surges in human density during scheduled inter-class intervals and lunch recesses. In typical collegiate schedules, thousands of students and faculty members transition simultaneously into dining commons during brief 10 to 20-minute passing periods. Under classical queuing theory, when customer arrival rates momentarily eclipse counter service capacity, queue lengths expand non-linearly, giving rise to protracted wait intervals, delayed classroom attendance, and chaotic vendor counters.

Commercial food delivery ecosystems, including Uber Eats, DoorDash, Swiggy, and Zomato, are architecturally ill-suited for dense institutional micro-environments. First, their revenue models impose vendor commissions ranging from 18% to 30%, which eliminates the viable margins of small campus kiosks and affordable student meal plans. Second, third-party courier dispatch is physically barred from entering institutional academic blocks and inner quadrangles. Third, commercial apps lack physical perimeter enforcement; remote students frequently trigger orders while miles away, leading to cold food accumulation, unclaimed trays, and counter bottlenecks.

To address these deficiencies, we developed SkipQ—a real-time micro-ordering and queue mitigation system built natively for campus dining. SkipQ shifts the order placement and kitchen fulfillment paradigms through five distinct engineering innovations:

- **Perimeter Geofencing:** Uses a client-side Haversine calculation with a 500-meter ordering limit and a bounded location-accuracy tolerance.
- **Buyer and Seller Workflows:** Connects menu browsing, cart placement, seller queue management, order-state updates, and pickup completion.
- **Dynamic Booking Pass:** Provides a persistent active-order indicator, a digital pickup pass, status tracking, and audio or haptic feedback where supported.
- **Pickup PIN Verification:** Uses a four-digit pickup PIN as an operational handover check at the seller POS.
- **Anti-Ghosting Controls:** Records unclaimed-order handling and account restrictions; production enforcement depends on deployed backend endpoints and rules.

II. THEORETICAL FOUNDATION & RELATED WORK

I. Queuing Dynamics in Dense Campuses

Cafeteria service counters represent an M/M/c queuing system with Poisson arrivals at rate λ and exponential service times at rate μ distributed across c cashiers. In a conventional manual queue, total time spent by a student W is given by Little's Law:

$$W = W_q + 1/\mu = \lambda / [\mu(\mu - \lambda)] + 1/\mu \quad (1)$$

As λ approaches service capacity, counter dwell times diverge asymptotically towards infinity. SkipQ decouples the ordering and payment phase from physical counter arrival, converting synchronized arrival bursts into an asynchronous preparation queue. The magnitude of any queue-time improvement must be established through measurement rather than assumed from the architecture alone.

II. Geodetic Perimeter Calculation

To guarantee proximity without straining backend server compute with continuous telemetry tracking, SkipQ evaluates user coordinates client-side against predefined university geolocation coordinates using the spherical earth Haversine equation:

$$a = sin^2(\Delta\phi/2) + cos(\phi_1)cos(\phi_2)sin^2(\Delta\lambda/2) \quad (2)$$

$$d = 2R \cdot atan2(\sqrt{a}, \sqrt{(1 - a)}) \quad (3)$$

where ϕ represents geodetic latitude, λ represents longitude, and R is the mean Earth radius taken as 6,371,000 meters. The current implementation permits ordering when the distance, adjusted by a bounded GPS-accuracy tolerance, is within 500 meters of the selected canteen.

III. SYSTEM ARCHITECTURE & DESIGN

SkipQ is an Expo and React Native application targeting Android, iOS, and web environments. Its architecture separates the client presentation layer, local application state, optional Cloud Firestore synchronization, and deployable backend and security controls.

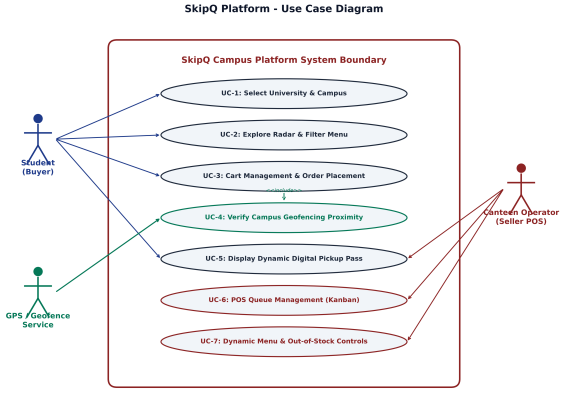


FIGURE 1. SYSTEM USE CASE MODEL. Functional interactions illustrating student buyers exploring menus within geofenced boundaries, vendor POS operations, and automated background transaction handling.

I. Presentation Tier & Dynamic Booking Pill

The buyer interface implements a Campus Radar view that dynamically groups food stalls by walking distance, current queue load, and operational status. Upon order commitment, the client spawns an isolated background timer component, the Dynamic Active Booking Pill. Inspired by modern dynamic island hardware primitives, this persistent HUD element renders real-time state progressions (Placed → Preparing → Ready) without obstructing subsequent catalog browsing.

II. Vendor POS Kitchen Controller

The vendor interface is optimized for high-throughput touch-screen kitchen displays. It aggregates live inbound tickets categorized into 'Pending Acceptance', 'In Kitchen', and 'Ready for Handoff'. The POS system leverages browser Web Audio synthesizer primitives and HTML5 Speech Synthesis APIs to broadcast multi-frequency acoustic buzzers and speech token announcements when orders transition to 'Ready', eliminating kitchen worker display distractions.

IV. IMPLEMENTATION & CORE SUBSYSTEMS

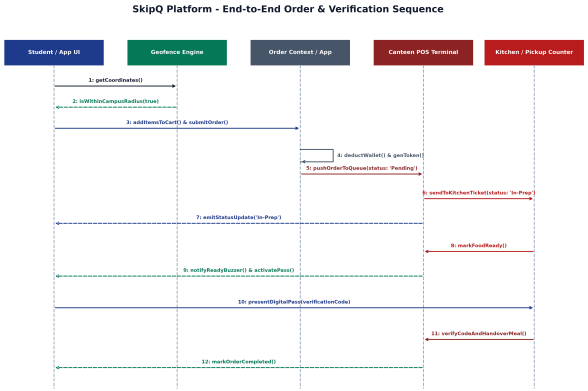


FIGURE 2. END-TO-END ORDER LIFECYCLE SEQUENCE. Chronological interaction sequence depicting geofence validation, optimistic UI updates, Firestore state replication, POS kitchen acknowledgement, and 4-digit PIN verification.

I. Synchronization and Local Persistence

When Firebase is configured, SkipQ uses Cloud Firestore listeners for order and canteen updates. The web build additionally uses BroadcastChannel, with a localStorage event fallback, to notify other open browser tabs. AsyncStorage preserves selected local application state on device. These mechanisms improve responsiveness but do not substitute for server-side validation of financial or order-state transitions.

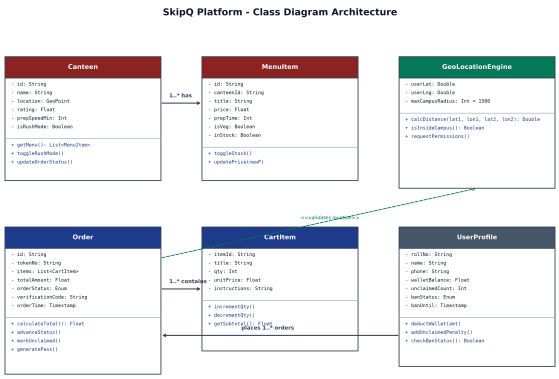


FIGURE 3. DOMAIN ENTITY & ARCHITECTURAL CLASS DIAGRAM. Comprehensive structural model detailing Canteen, MenuItem, Order, DigitalPass, and UserProfile entities coordinating state across client and Firebase tiers.

II. Anti-Ghosting & Abandonment Policy

Unclaimed meals can create avoidable waste for canteen operators. SkipQ contains client-side handling for abandoned orders, wallet adjustments, and account restrictions. For production use, these sensitive operations must be mediated by the deployed backend endpoints and Firestore rules:

- **Refund and penalty policy:** Define business rules and make the deployed backend the authority for wallet transfers and refunds.
- **Unclaimed-order tracking:** Record order status and an unclaimed-order count through validated server-side operations.
- **Account restrictions:** Apply lockouts only after a documented threshold and a user-facing appeal or support process.

V. IMPLEMENTATION VALIDATION PLAN

The repository contains component-level tests for distance calculations and wallet or order behaviour, but it does not include a completed field-study dataset. Before making operational performance claims, a future study should define a baseline, obtain appropriate institutional approval, collect anonymized measurements, and report sample size, timing, failure cases, and statistical treatment.

TABLE 1. IMPLEMENTATION COVERAGE AND VALIDATION EVIDENCE.

Area	Implemented	Current evidence	Field metric
Geofence	Haversine + tolerance	distance unit test	To measure
Order lifecycle	Buyer, POS, pickup PIN	order/wallet test	To measure
Real-time updates	Firestore; web tab	implementation review	To measure
Persistence	Firestore / AsyncStorage	implementation review	To measure
Security controls	Rules + Functions available	deployment checklist	To validate

Table 1 separates implemented capabilities from claims that require future measurement. A pilot evaluation should capture order placement time, preparation and pickup duration, abandoned orders, synchronization delay, and seller throughput, then compare these measures with a documented manual-queue baseline.

VI. DISCUSSION & SECURITY CONSIDERATIONS

The application performs proximity checks on the client before allowing cart and order actions. Client-side checks improve user feedback but are not a complete security boundary; a production deployment should validate privileged order, payment, refund, and seller actions through deployed Cloud Functions and restrictive Firestore rules.

The repository includes a deployment guide that requires seller claims to be issued through a trusted administrative process and recommends deploying the order endpoints and Firestore rules before live ordering is enabled. These controls must be configured and tested in the target Firebase project; their presence in source control alone is not proof of a secure production deployment.

VII. CONCLUSION & FUTURE WORK

This paper presented SkipQ, an Expo-based campus dining prototype that combines client-side proximity checks, buyer and seller ordering workflows, real-time Firestore integration when configured, and a pickup-PIN handover flow. The implementation is positioned for controlled evaluation; it does not establish queue reduction, abandonment, latency, or throughput outcomes without a documented study.

Future engineering initiatives include integrating machine-learning-driven kitchen preparation time estimators based on real-time grill load, integrating RFID/NFC smart student ID cards for zero-touch physical pickup lockers, and extending the multi-tenant architecture across affiliated state university campuses.

ACKNOWLEDGEMENTS

The authors express sincere gratitude to the Department of Computer Engineering and the administration of Silver Oak University for providing institutional testing facilities, network access, and direct collaboration with campus dining operators.

REFERENCES

[1] D. Gross, J. F. Shortle, J. M. Thompson, and C. M. Harris, *Fundamentals of Queueing Theory*, 5th ed. Hoboken, NJ: John Wiley & Sons, 2018.

[2] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[3] Firebase, "Get data with Cloud Firestore: Listen to realtime updates," Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/firestore/query-data/listen

[4] Firebase, "Get started with Cloud Firestore Security Rules," Firebase Documentation. [Online]. Available: https://firebase.google.com/docs/firestore/security/get-started

[5] Expo, "Expo SDK 57 Reference: Location," Expo Documentation. [Online]. Available: https://docs.expo.dev/versions/v57.0.0/sdk/location/

[6] W3C, "Web Audio API," W3C Recommendation. [Online]. Available: https://www.w3.org/TR/webaudio/

ABOUT THE AUTHORS

Satyam Patel is an Undergraduate Researcher in the Department of Computer Engineering at Silver Oak University, Ahmedabad, India. His research interests include distributed mobile systems, edge computing, real-time reactive architectures, and campus automation.

Harsh V. Trivedi is an Undergraduate Scholar in the Department of Computer Engineering at Silver Oak University. His focus areas include mobile application engineering, geofencing protocols, and user experience telemetry.

Priya R. Shah is an Assistant Professor in the Department of Information Technology at Silver Oak University. Her research spans cloud computing, queuing theory, distributed transaction systems, and educational technology infrastructure.